

The Impact of Register Allocation on ILP

In my last two articles, I had explained in detail the concepts of Instruction Level Parallelism (ILP) and Register Allocation. In this article, we will look at how ILP is closely linked with register allocation, and explore three methods of applying them to code: register allocation before instruction scheduling, after scheduling, and hand-in-hand (together).

Achieving ILP involves the scheduling of instructions—i.e., re-ordering instructions in order to combine the instructions in parallel. Thus, we may define instruction scheduling as the process of taking the code generated by the code-generation module (or register allocation module) of the compiler, and re-ordering the code such that a more efficient schedule is produced.

Please note that in this article, the term instruction scheduling should be understood to mean instruction scheduling for ILP.

ILP and register allocation

Register allocation replaces the virtual registers assigned by the code-generation module with processor (or physical) registers. The most widely used method for register allocation is colouring the interference graph, but this method may not produce efficient code for processors that support instruction-level parallelism. As we will see, it may introduce a false dependency between two instructions, and restrict ILP.

In processors that support ILP, generally there are two separate modules for register allocation and instruction scheduling for ILP.